Community Library System: Class Exercise

Scenario

A small community library still tracks its books and loans on paper. You'll build a Windows desktop app so staff can manage the **book catalogue** and record **loans** to borrowers. You'll build the database, write stored procedures, create custom classes that call them, and use those classes from your forms.

Part 1: Database Setup

Create a database called LibraryDB with two tables.

Books

Column	Type	Notes
BookID	INT	Primary key, identity
Title	NVARCHAR(150)	Required
Author	NVARCHAR(100)	Required
Genre	NVARCHAR(50)	e.g. "Fiction", "Science"
TotalCopies	INT	Must be greater than 0

Loans

Column	Type	Notes
LoanID	INT	Primary key, identity
FirstName	NVARCHAR(50)	Borrower's first name, required
LastName	NVARCHAR(50)	Borrower's last name, required
Phone	NVARCHAR(20)	
BookID	INT	Foreign key to Books
LoanDate	DATE	
DueDate	DATE	Must be after LoanDate

Insert at least 3 sample books and 5 sample loans using a script.

Part 2: Stored Procedures

Write **all** database access as stored procedures.No inline SQL in C# code.

For Books:

1. sp_AddBook
2. sp_GetAllBooks
3. sp_UpdateBook
4. sp_DeleteBook

For Loans:

1. sp_AddLoan
2. sp_GetAllLoans (JOIN with Books to show the book title)
3. sp_UpdateLoan
4. sp_DeleteLoan
5. sp_SearchLoans, which takes a search term and matches borrower first name, last name, or LoanID

Test each procedure in SSMS using EXEC **before** writing any C#.

Part 3: Custom Classes

Add two classes to your project: Book and Loan. Each class represents one table and is responsible for talking to the database.

Each class should have:

- Properties matching the table columns (e.g. BookID, Title, Author...)
- A connection string
- Methods that call the stored procedures

Suggested methods for Book:

Method	Returns	Calls
Add()	void	sp_AddBook
Update()	void	sp_UpdateBook
Delete()	void	sp_DeleteBook
GetAll()	DataTable	sp_GetAllBooks

Suggested methods for Loan: the same four, plus Search(string term) returning a DataTable.

Each method must use:

- SqlConnection and SqlCommand with CommandType.StoredProcedure
- Parameters.AddWithValue or Parameters.Add
- SqlDataAdapter and DataTable for the methods that return data

The Add, Update and Delete methods use the object's own properties as parameter values, so the form creates an object, sets its properties, and calls the method.

Part 4: Login Form

- Form with Username and Password textboxes and a Login button.
- Validate that neither field is empty.
- Check the credentials against a users.txt file (one username,password per line). Place the file in the same folder as the .exe (bin\Debug).
- On success, open the main form. On failure, show a message.
- Mask the password field (UseSystemPasswordChar = true).

Part 5: Main Form

One form with a **TabControl** containing two tabs. All the code lives in this form's event handlers and calls your Book and Loan classes. There is no database code in the form itself.

Books tab

- DataGridView showing all books
- Textboxes for the book fields
- Buttons: Add, Update, Delete, Clear
- Clicking a grid row fills the textboxes

Loans tab

- DataGridView showing all loans
- Textboxes for borrower details and two DateTimePickers (loan date, due date)
- A ComboBox filled with the books (use Book.GetAll() and set DisplayMember/ValueMember)
- Buttons: Add, Update, Delete, Clear
- Search textbox, Search button, and a "Show All" button

Part 6: Validation and Error Handling

- Required fields must not be empty.
- Total copies must be a positive whole number (int.TryParse).
- Due date must be after loan date.
- Wrap calls in try/catch (SqlException) and show friendly messages.
- Ask for confirmation before deleting (MessageBox with Yes/No).
- Handle the case where a book can't be deleted because loans exist for it.

Checklist for Students

- Database and both tables created with sample data
- All 9 stored procedures tested in SSMS
- Book and Loan classes created with properties and methods
- Login works using users.txt
- Books: add, view, update, delete working
- Loans: add, view, update, delete working
- Loan search works

- No SQL or SqlCommand code in the forms, and no inline SQL anywhere
- Validation and try/catch in place

Optional Practice

1. Add a stored procedure and button that lists all loans for a selected book.
2. Add a "Show Overdue Loans" button
3. Create a custom exception such as InvalidLoanDatesException and throw it from the Loan class.
4. Add a failed-login counter that closes the app after 3 attempts.